BRi : services dynamiques

R5A08 - Qualité de développement | Présentation technique
État vérifié le 8 novembre 2026

1. Objectif et périmètre réalisé

La plateforme met en relation des programmeurs Java et des utilisateurs amateurs. Le serveur BRiLaunch reçoit les deux publics sur des ports distincts. Un programmeur inscrit fournit l'adresse de son répertoire FTP et peut installer ou mettre à jour une classe de service pendant que le serveur reste en fonctionnement.

Le périmètre vise le cahier des charges minimum : services sans ressources partagées, échangeant des textes. Trois exemples sont fournis au démarrage : inversion d'un texte, conversion en majuscules et comptage de mots. Le client amateur et le client programmeur sont deux applications console distinctes.

Exigence obligatoire	Réalisation
Deux ports et deux applications clientes	BRiLaunch : 3000 / 4000 ; ClientAma et ClientProg
Compte programmeur et changement FTP	Login/mot de passe, certification automatique, validation du login et de l'URL
Installation depuis le FTP	Téléchargement binaire de la classe, contrôle BRi et contrôle du package
Mise à jour sans redémarrage	Nouveau chargeur par version ; remplacement atomique au même numéro
Choix et exécution d'un service amateur	Menu dynamique ; lancement avec la socket de la session

2. Organisation du code

src_serveur/bri contient les listeners, les gestionnaires de sessions, les registres et le chargeur FTP. **src_client** contient les deux clients et les trois services d'exemple. **test** contient des tests autonomes sans bibliothèque Java externe. Les scripts fournis construisent le projet avec un JDK 17 ou supérieur, en ciblant Java 17. Le README donne aussi les commandes javac directes et les étapes d'import Eclipse.

Les extensions JAR, les services avec fichiers ou ressources partagées, la messagerie, l'envoi de courriels et les commandes explicites démarrer/arrêter/désinstaller sont optionnels dans l'énoncé et ne sont pas réalisés.

Choix de développement

3. Chargement réellement distant

Un nom tel que alice.MonService correspond au fichier alice/MonService.class sous le répertoire FTP déclaré. Chaque installation ou mise à jour utilise un nouveau FtpServiceLoader. Les classes du package du programmeur sont téléchargées directement : aucune classe déjà présente dans le classpath du serveur ne peut masquer un FTP inaccessible. L'interface bri.Service conserve en revanche le chargeur de la plateforme afin de préserver son identité Java.

Ce choix évite le défaut d'un URLClassLoader parent-first qui pourrait retrouver une ancienne classe locale. Un nouveau chargeur donne aussi une nouvelle identité à chaque version : les futures sessions voient la nouvelle version tandis qu'une session déjà commencée termine avec son ancienne classe. Les ressources sont libérées lorsque les anciennes classes et leur chargeur ne sont plus référencés.

4. Validation et remplacement atomique

Le registre exige une classe publique, concrète, implémentant bri.Service, un constructeur public(Socket) et une méthode déclarée public static String toStringue(). La description est évaluée avant installation et doit tenir sur une ligne non vide. Le package doit être exactement le login du programmeur. Les règles implémentées correspondent au contrat BRi disponible dans le dépôt ; d'éventuelles règles supplémentaires du TP5 restent à comparer au support de cours non fourni.

Chaque entrée conserve sa classe, sa description et son propriétaire. Une installation en double est refusée ; la commande de mise à jour exige une entrée existante appartenant au même programmeur. Tous les contrôles précèdent la modification du registre. Une classe invalide, un téléchargement échoué ou un bytecode corrompu laisse donc la version précédente disponible. Le numéro du service est conservé.

5. Sockets, protocole et concurrence

Les messages sont des lignes UTF-8. Un menu se termine par une ligne vide. Le client amateur choisit un numéro, attend la consigne du service, envoie son texte puis lit le résultat. Le client programmeur envoie un numéro d'action, le login, le mot de passe et une valeur : URL FTP pour inscription/changement, nom complet de classe pour installation/mise à jour. Chaque connexion réalise une action et reçoit une réponse OK ou ERROR.

Les acceptations et sessions s'exécutent dans des tâches concurrentes. Les opérations de compte et de registre sont synchronisées, notamment pour empêcher deux inscriptions simultanées avec le même login. Les sessions ont un délai de lecture de 30 secondes ; les téléchargements ont des délais de connexion/lecture de 5 secondes et une limite de 4 Mio par classe. La fermeture du serveur ferme les listeners et les sockets actives. Les deux ports sont réservés avant de démarrer les tâches : un échec de bind ne laisse pas un demi-serveur.

Validation et utilisation

6. Résultats de vérification

La commande **test.cmd** sous Windows (ou ./test.ps1) compile avec --release 17 -Xlint:all et lance les trois exécutables de tests. Résultat observé : **110 vérifications réussies**, sans avertissement de compilation. Les tests utilisent leurs propres ports locaux et n'exigent ni accès Internet ni serveur FTP externe.

Test	Nombre	Ce qui est vérifié
ProgrammerAccountTest	38	Validation, authentification, changement FTP, inscription concurrente en double
AmateurFlowTest	16	Services texte, Unicode, texte vide, saisies invalides, abandon, concurrence
PlatformFlowTest	56	Vrais handlers et clients, FTP binaire, installations et mises à jour en fonctionnement

Le test de plateforme compile un service hors du classpath applicatif et le distribue via un petit serveur FTP de test. Il installe v1, ouvre une session, publie v2 sans redémarrer, vérifie v2 pour un nouveau client et v1 pour l'ancienne session. Il change ensuite de répertoire FTP et publie v3. Il contrôle aussi les mots de passe/packages incorrects, doublons, classes non conformes, bytecode corrompu, requêtes tronquées, propriété des services et indisponibilité du FTP.

7. Démonstration reproductible

Exécuter **build.cmd** sous Windows, puis **java -cp build/classes bri.BRiLaunch**. Dans deux autres terminaux, lancer respectivement **clientama.ClientAma localhost 3000** et **clientprog.ClientProg localhost 4000** avec le même préfixe java -cp build/classes. Le README décrit aussi la compilation sans script, le dépôt sur un vrai FTP et les actions du client programmeur. Les clients se relancent pour chaque nouvelle action.

8. Limites et améliorations possibles

Les comptes et services installés sont en mémoire et repartent à zéro au redémarrage. Les mots de passe sont conservés en mémoire ; les protocoles FTP et socket de l'exercice ne sont pas chiffrés. Les services s'exécutent dans la JVM du serveur et doivent être de confiance. Le pool de sessions n'est pas borné. Les délais réseau limitent une lecture bloquée, mais pas la durée totale d'un transfert qui progresse lentement. Toutes les configurations de serveurs FTP externes n'ont pas été testées.

Les améliorations possibles sont la persistance, le stockage dérivé des mots de passe, un transport protégé, une isolation des services et un pool borné. Ces points ne sont pas présentés comme déjà réalisés. Les tests étayent le scénario pédagogique minimum ; ils ne garantissent pas l'absence absolue de défauts pour tout service tiers.

Remise : package.ps1 produit une archive du code source et de ce rapport sous la limite de 2 Mo. Utiliser -GroupName avec les noms réels des membres du groupe avant le dépôt Moodle. Les dépendances et classes compilées sont exclues ; les fichiers d'import Eclipse sont inclus.